CI/CD Pipeline Implementation using Jenkins, Docker & Kubernetes

WINTER SEMESTER 2025-26

ASSESSMENT - 5

Name: P.Jaswanth

Registration ID: 23MIS0178

School: School of Computer Science Engineering and Information Systems

Course Code: ISWE406P

Course Title: Agile Development Process And Devops Lab

Faculty: Dr. Ramalingam M

Slot: L57+L58

Table of Contents

1. Introduction
2. Lab 1: Simple Java CI/CD Application
   2.1 Project Overview
   2.2 Application Source Code (App.java)
   2.3 Maven Build Configuration (pom.xml)
   2.4 Dockerfile
   2.5 Kubernetes Deployment (deployment.yaml)
   2.6 Jenkins Pipeline
   2.7 Pipeline Execution Screenshots
3. Lab 2: Spring Boot CI/CD Application
   3.1 Project Overview
   3.2 Application Source Code (App.java & HelloController.java)
   3.3 Maven Build Configuration (pom.xml)
   3.4 Application Properties
   3.5 Dockerfile
   3.6 Kubernetes Deployment (deployment.yaml)
   3.7 Jenkins Pipeline
   3.8 Pipeline Execution Screenshots
4. Summary & Conclusion

1. Introduction

This report documents the implementation of two CI/CD pipeline labs using an Agile DevOps toolchain. Both labs demonstrate the end-to-end automation of building, containerising, and deploying Java-based applications to a Kubernetes cluster via Jenkins.

The key technologies used across both labs are:

- Java 17 — Application runtime and development language
- Maven — Build automation and dependency management
- Docker — Containerisation of the application
- DockerHub — Container image registry
- Jenkins — CI/CD automation server (pipeline as code)
- Kubernetes (kubectl) — Container orchestration and deployment

2. Lab 1: Simple Java CI/CD Application

2.1 Project Overview

This lab implements a minimal Java application that continuously prints a message to the console. The objective is to set up a full CI/CD pipeline from source code on GitHub all the way to a running container in Kubernetes, automated entirely through Jenkins.

The application runs an infinite loop, printing status messages every 5 seconds — simulating a long-running service.

2.2 Application Source Code (App.java)

The main application class is a simple Java program with a continuous loop:

```
public class App {
    public static void main(String[] args) throws Exception {
        System.out.println("Java CI/CD Application Started...");
        while (true) {
            Thread.sleep(5000);
            System.out.println("Hello from Java CI/CD Pipeline!");
        }
    }
}
```

2.3 Maven Build Configuration (pom.xml)

The Maven POM file defines the project build lifecycle using the Compiler Plugin (targeting Java 17) and the JAR Plugin to produce an executable JAR with the correct main class manifest entry:

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.demo</groupId>
  <artifactId>simple-java-app</artifactId>
  <version>1.0</version>
  <build>
    <plugins>
      <!-- Compiler Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.10.1</version>
        <configuration>
          <source>17</source>
          <target>17</target>
        </configuration>
      </plugin>
      <!-- JAR Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.3.0</version>
        <configuration>
          <archive>
            <manifest>
              <mainClass>App</mainClass>
```

```
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

2.4 Dockerfile

The Dockerfile uses the official Eclipse Temurin JDK 17 base image, copies the compiled JAR into the container, and sets the default command to run the application:

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY target/simple-java-app-1.0.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

2.5 Kubernetes Deployment (deployment.yaml)

The Kubernetes manifest defines a Deployment with 2 replicas and a NodePort Service to expose the application externally on port 30008:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
      containers:
      - name: java-app
        image: pothurujaswanth/java-app:latest
---
apiVersion: v1
kind: Service
metadata:
  name: java-service
spec:
  type: NodePort
  selector:
    app: java-app
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30008
```

2.6 Jenkins Pipeline

The Jenkins pipeline (Jenkinsfile) automates the full CI/CD flow in five stages: Clone → Build → Dockerize → Push → Deploy:

```
pipeline {
  agent any
  environment {
    DOCKER_USERNAME="pothurujaswanth"
    IMAGE_NAME="docker-project"
    IMAGE_TAG="latest"
  }
  stages {
    stage('Clone Repository') {
      steps {
        git branch: 'main',
            url:'https://github.com/jaswanthhitman45/maven.git'
      }
    }
    stage('Build with Maven') {
      steps {
        bat 'mvn clean package'
      }
    }
    stage('Build Docker Image') {
      steps {
        bat "docker build -t %DOCKER_IMAGE% ."
      }
    }
    stage('Push to DockerHub') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'dockerhub-cred',
          usernameVariable: 'DOCKER_USER',
          passwordVariable: 'DOCKER_PASS'
        )]) {
          bat """
            docker login -u %DOCKER_USER% -p %DOCKER_PASS%
            docker push %DOCKER_IMAGE%
          """
        }
      }
    }
    stage('Deploy to Kubernetes') {
      steps {
        bat 'kubectl apply -f deployment.yaml'
      }
    }
```

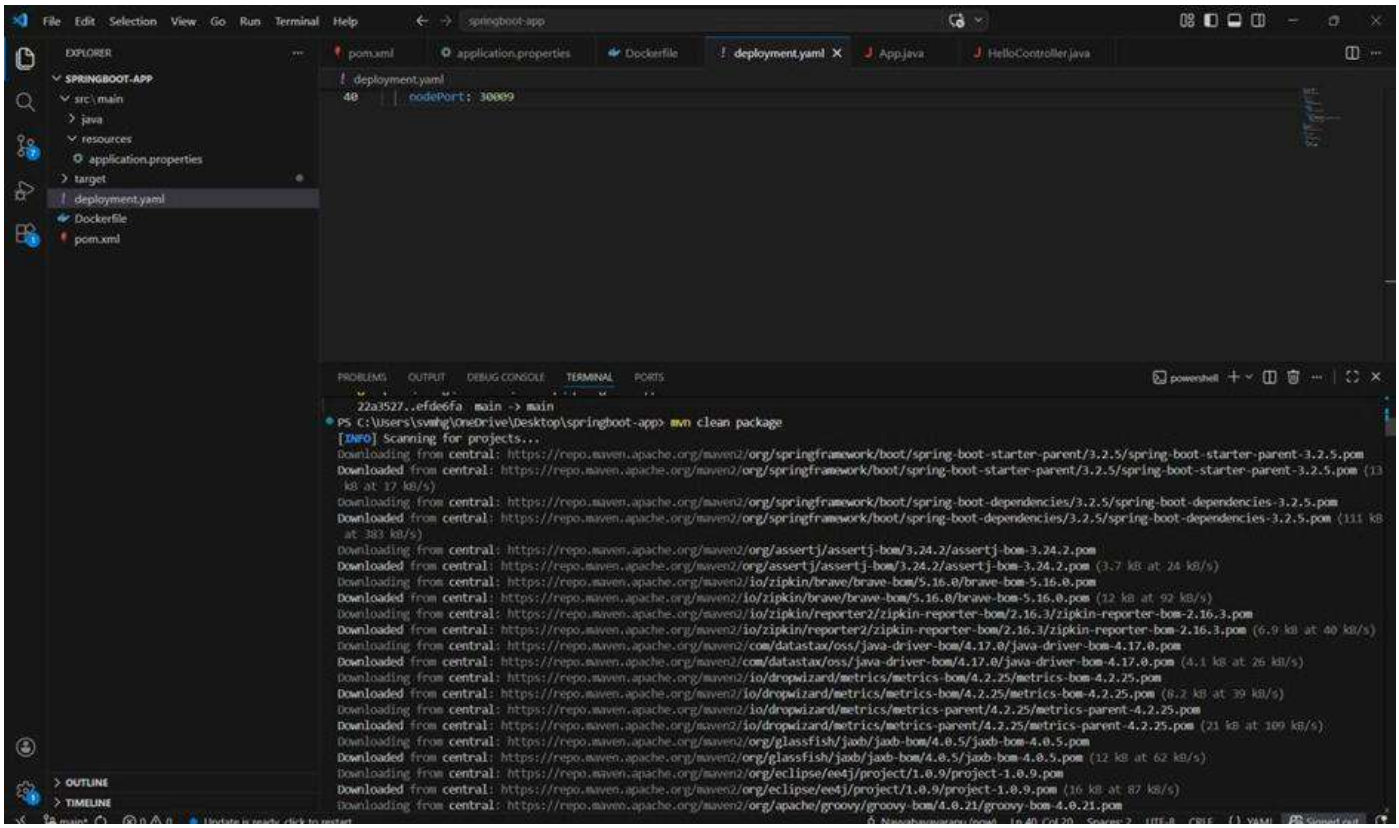
2.7 Pipeline Execution Screenshots

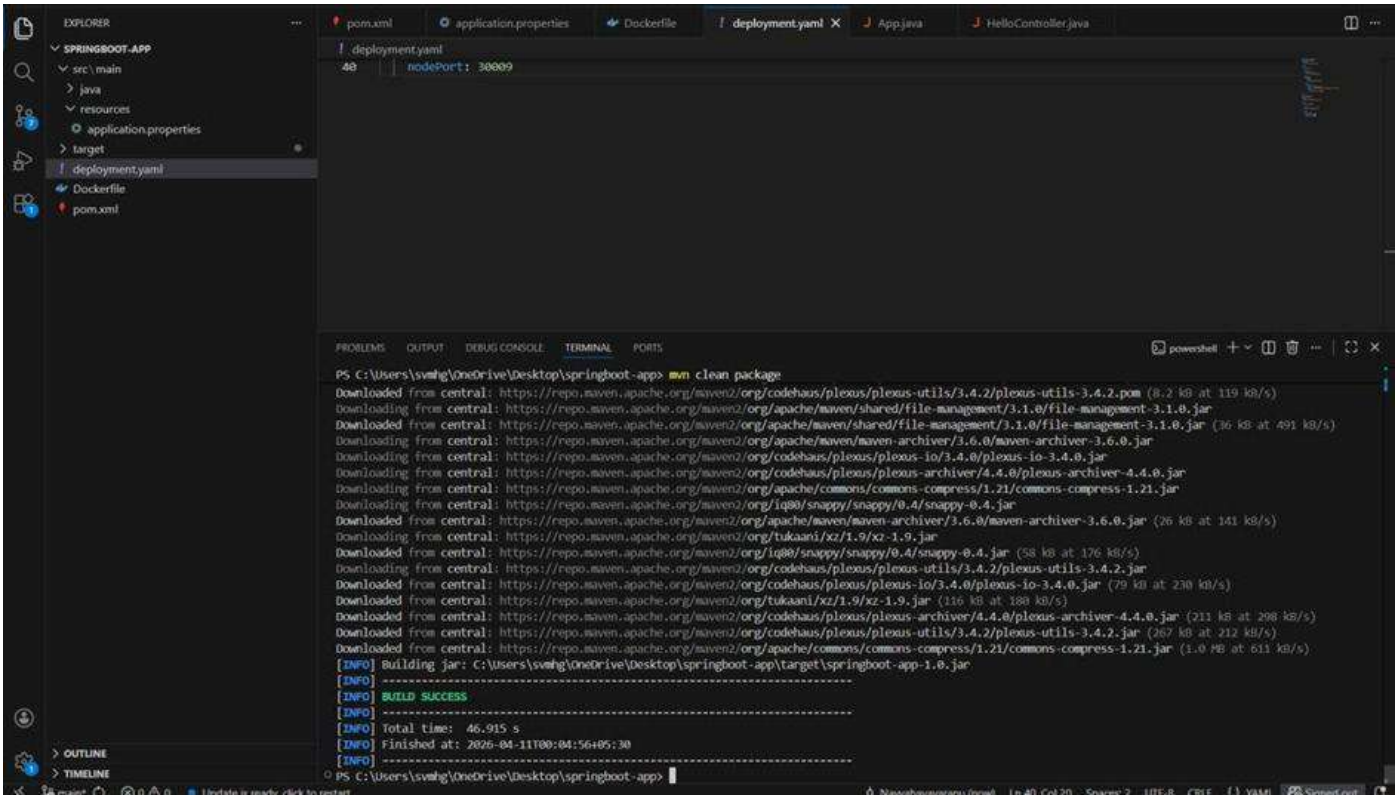
The following screenshots show the successful execution of each pipeline stage and the resulting deployment in Kubernetes.

Project directory

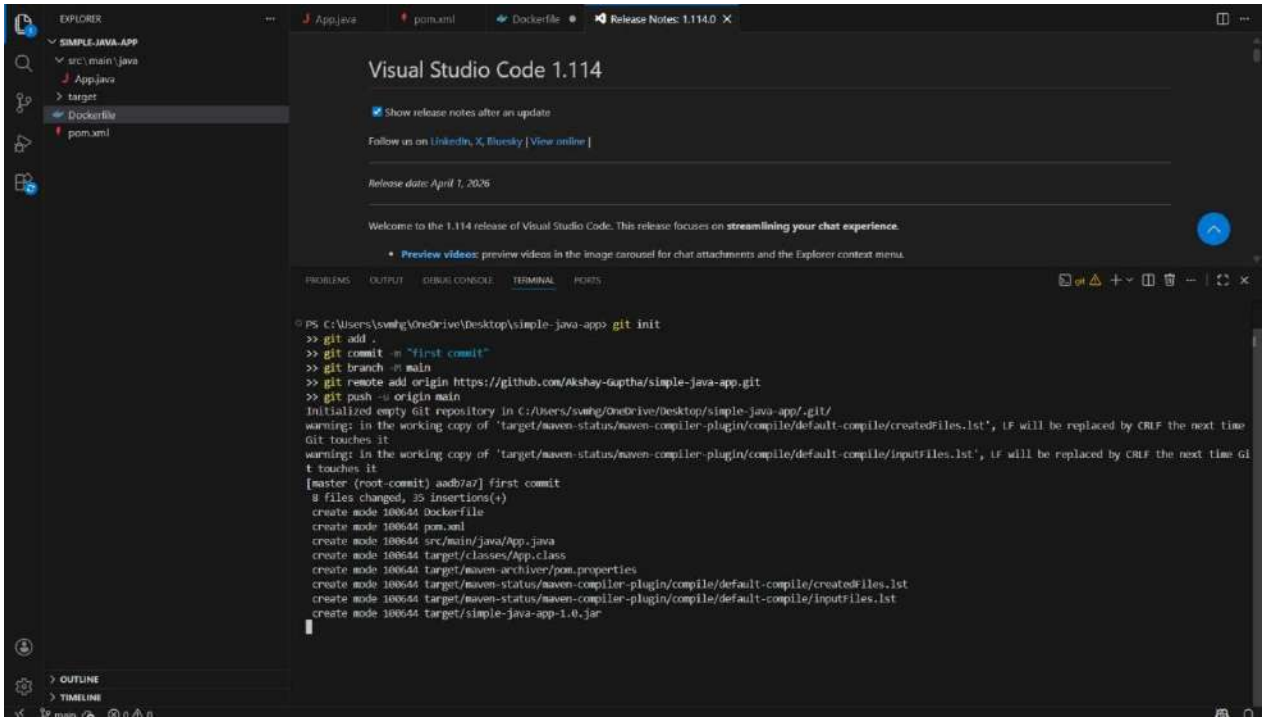
```
PS C:\Users\Lenovo\OneDrive\Desktop\agile> tree /f
    app.java
    deployment.yaml
    dockerfile
    pom.xml

    src
    └───main
        └───java
            └───com
                └───example
                        App.java

    target
    |   springboot-app-0.0.1-SNAPSHOT.jar
    |   springboot-app-0.0.1-SNAPSHOT.jar.original
    |
    ├───classes
    |   └───com
    |       └───example
    |               App.class
    |
    ├───generated-sources
    |   └───annotations
    ├───maven-archiver
    |       pom.properties
    |
    ├───maven-status
    |   └───maven-compiler-plugin
    |       └───compile
    |           └───default-compile
    |                   createdFiles.lst
    |                   inputFiles.lst
    |
    └───test-classes
```

Maven Build Output

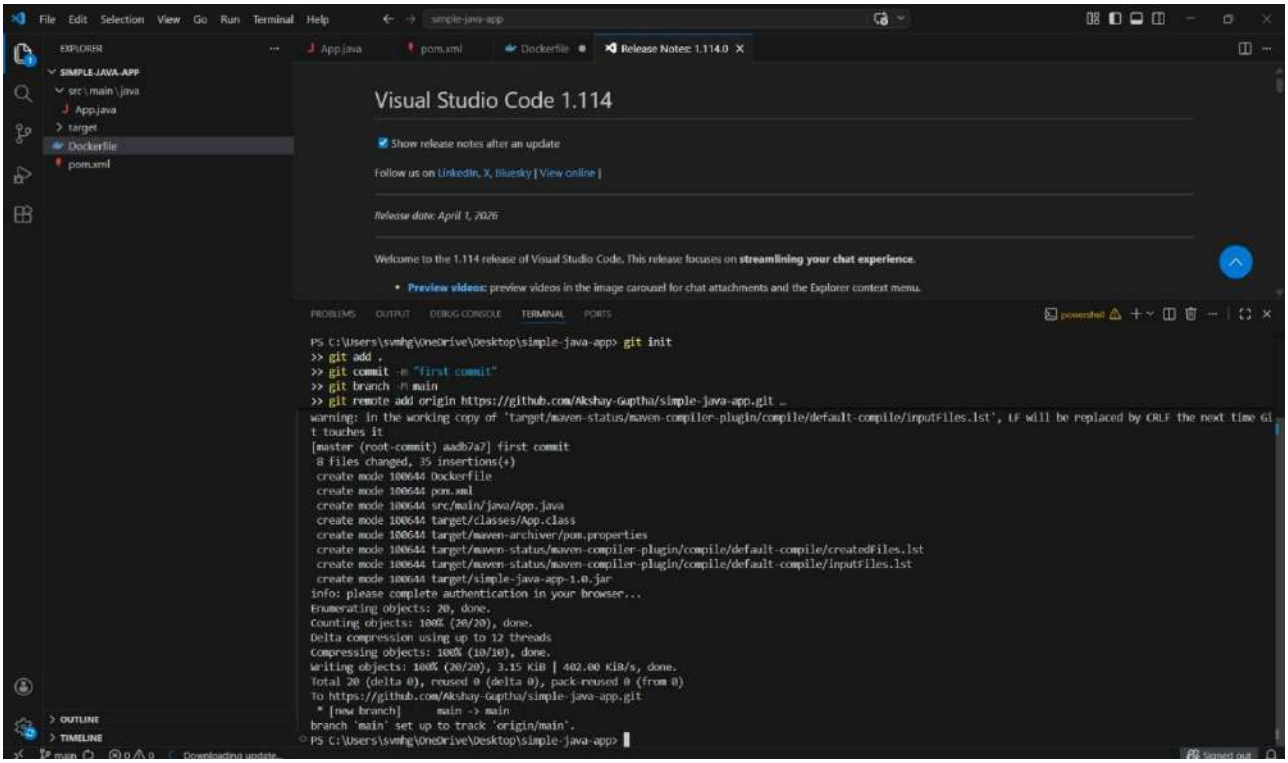

File Edit Selection View Go Run Terminal Help
springboot-app
EXPLORER
SPRINGBOOT-APP
src\main
java
resources
application.properties
target
deployment.yaml
Dockerfile
pom.xml
pom.xml
application.properties
Dockerfile
deployment.yaml
App.java
HelloController.java
deployment.yaml
40 nodePort: 30009
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
22a3527..efde6fa main -> main
PS C:\Users\svmhg\OneDrive\Desktop\springboot-app> mvn clean package
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-starter-parent/3.2.5/spring-boot-starter-parent-3.2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-starter-parent/3.2.5/spring-boot-starter-parent-3.2.5.pom (13 kB at 17 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-dependencies/3.2.5/spring-boot-dependencies-3.2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-dependencies/3.2.5/spring-boot-dependencies-3.2.5.pom (111 kB at 383 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/assertj/assertj-bom/3.24.2/assertj-bom-3.24.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/assertj/assertj-bom/3.24.2/assertj-bom-3.24.2.pom (3.7 kB at 24 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/io/zipkin/brave/brave-bom/5.16.0/brave-bom-5.16.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/io/zipkin/brave/brave-bom/5.16.0/brave-bom-5.16.0.pom (12 kB at 92 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/io/zipkin/reporter2/zipkin-reporter-bom/2.16.3/zipkin-reporter-bom-2.16.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/io/zipkin/reporter2/zipkin-reporter-bom/2.16.3/zipkin-reporter-bom-2.16.3.pom (6.9 kB at 40 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/datastax/oss/java-driver-bom/4.17.0/java-driver-bom-4.17.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/com/datastax/oss/java-driver-bom/4.17.0/java-driver-bom-4.17.0.pom (4.1 kB at 26 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-bom/4.2.25/metrics-bom-4.2.25.pom
Downloaded from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-bom/4.2.25/metrics-bom-4.2.25.pom (8.2 kB at 39 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-parent/4.2.25/metrics-parent-4.2.25.pom
Downloaded from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-parent/4.2.25/metrics-parent-4.2.25.pom (21 kB at 109 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/glassfish/jaxb/jaxb-bom/4.0.5/jaxb-bom-4.0.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/glassfish/jaxb/jaxb-bom/4.0.5/jaxb-bom-4.0.5.pom (12 kB at 62 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/eclipse/ee4j/project/1.0.9/project-1.0.9.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/eclipse/ee4j/project/1.0.9/project-1.0.9.pom (16 kB at 87 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/groovy/groovy-bom/4.0.21/groovy-bom-4.0.21.pom
OUTLINE
TIMELINE



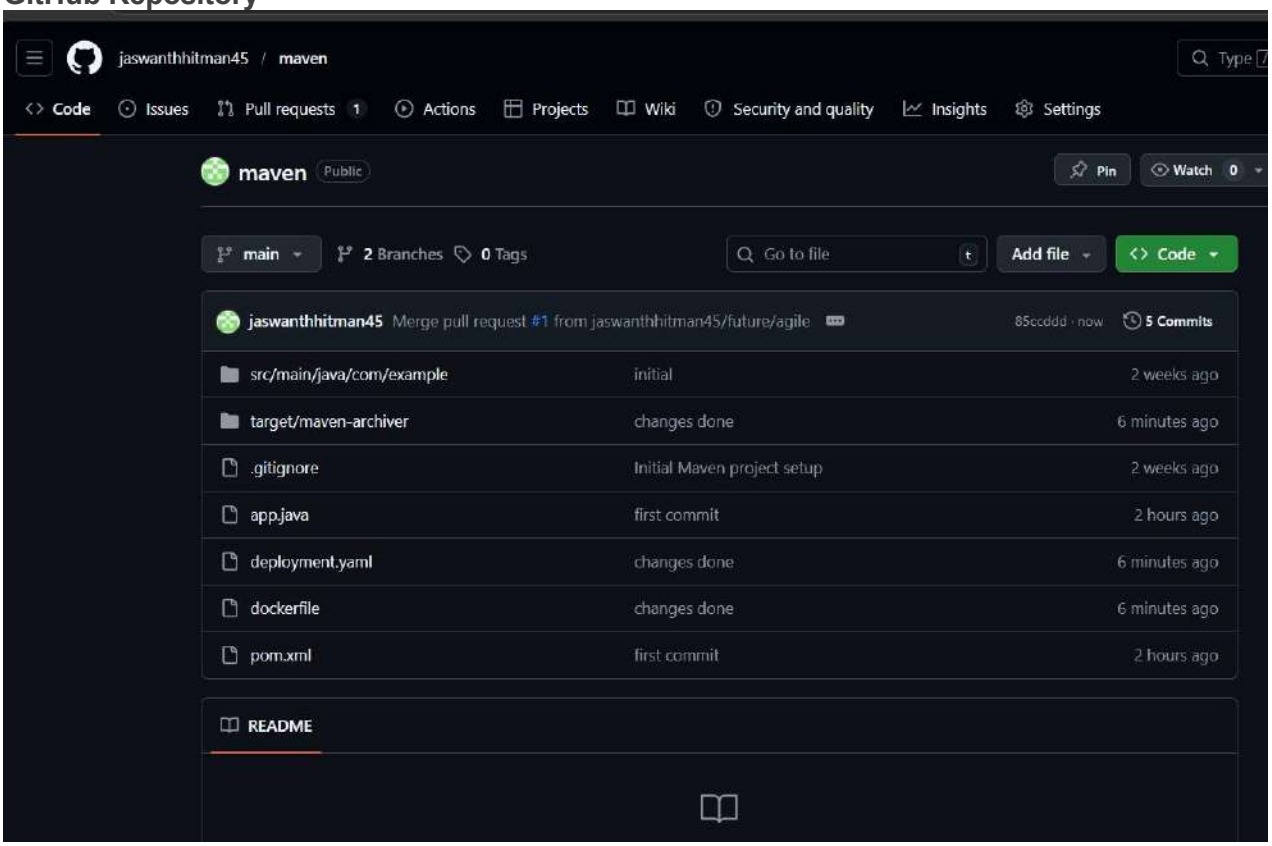
Git Push



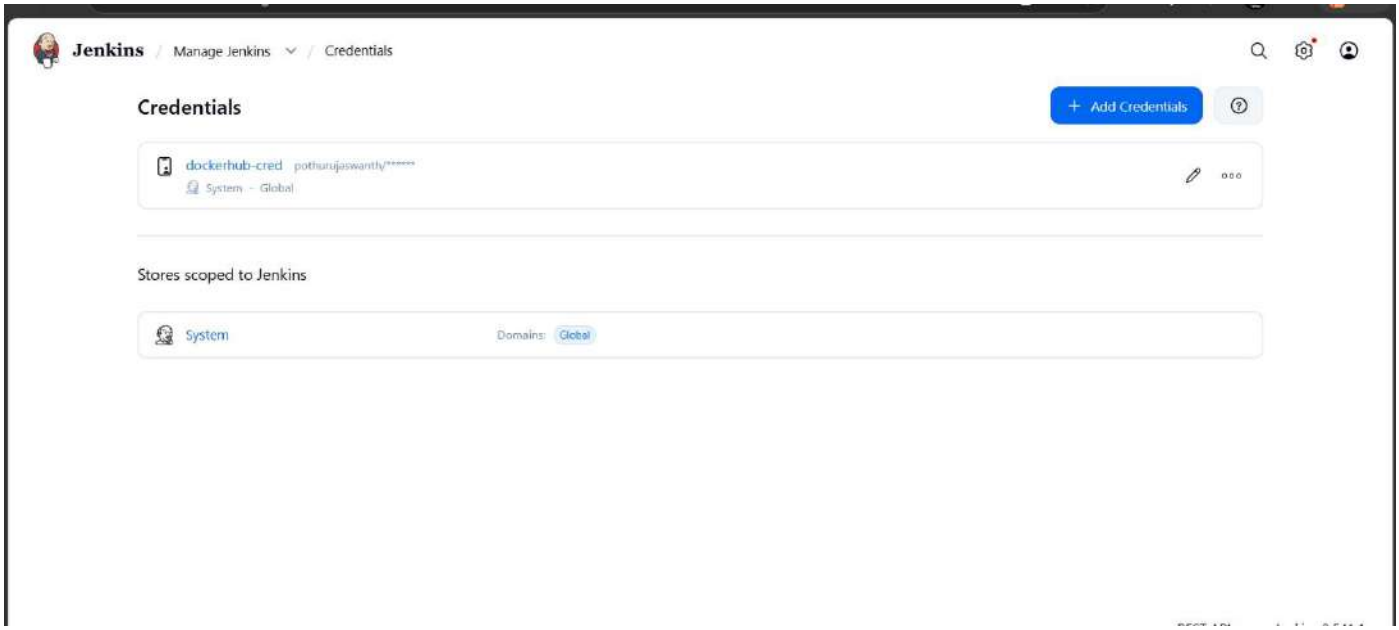
DockerHub Push



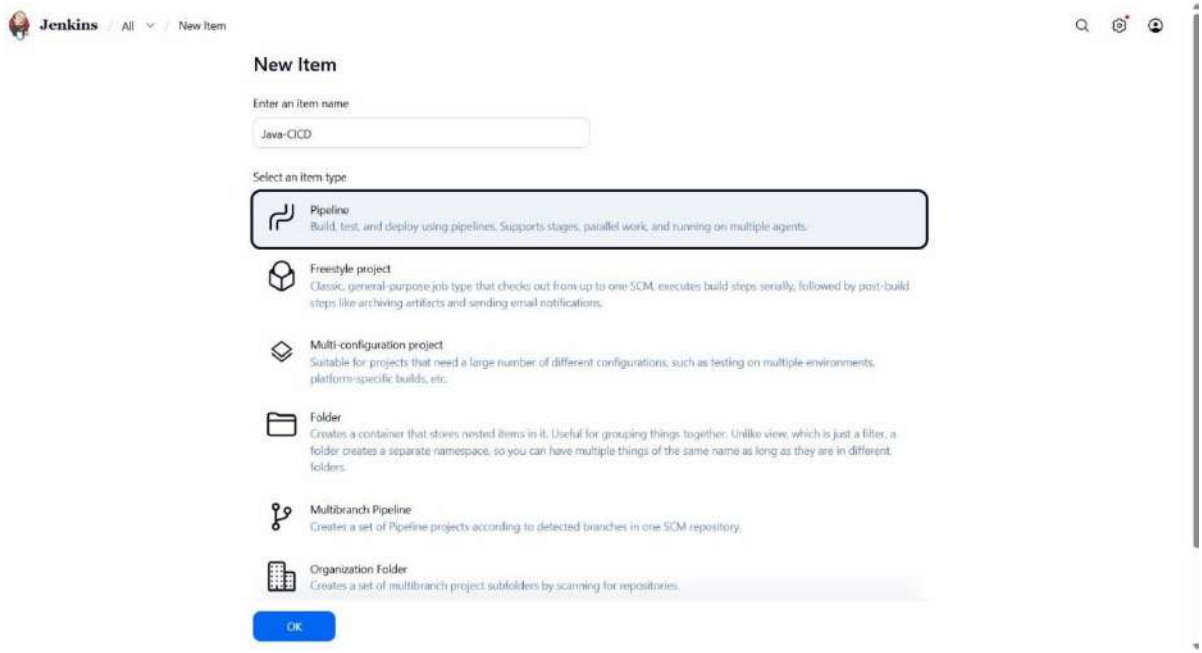
GitHub Repository



DockerHub Credentials



Pipeline Creation



Definition

Pipeline script

Script ?

try sample Pipeline...

```
    environment {
        DOCKER_USERNAME = "pothurujaswanth"
        IMAGE_NAME = "docker-project"
        IMAGE_TAG = "latest"
    }

    stages {
         stage('Clone GitHub Repo') {
    steps {
        git branch: 'main',
        url: 'https://github.com/jaswanthhitman45/maven.git'
    }
}
```

Save Apply

Jenkins Console Output

Jenkins / Java-CICD / #15

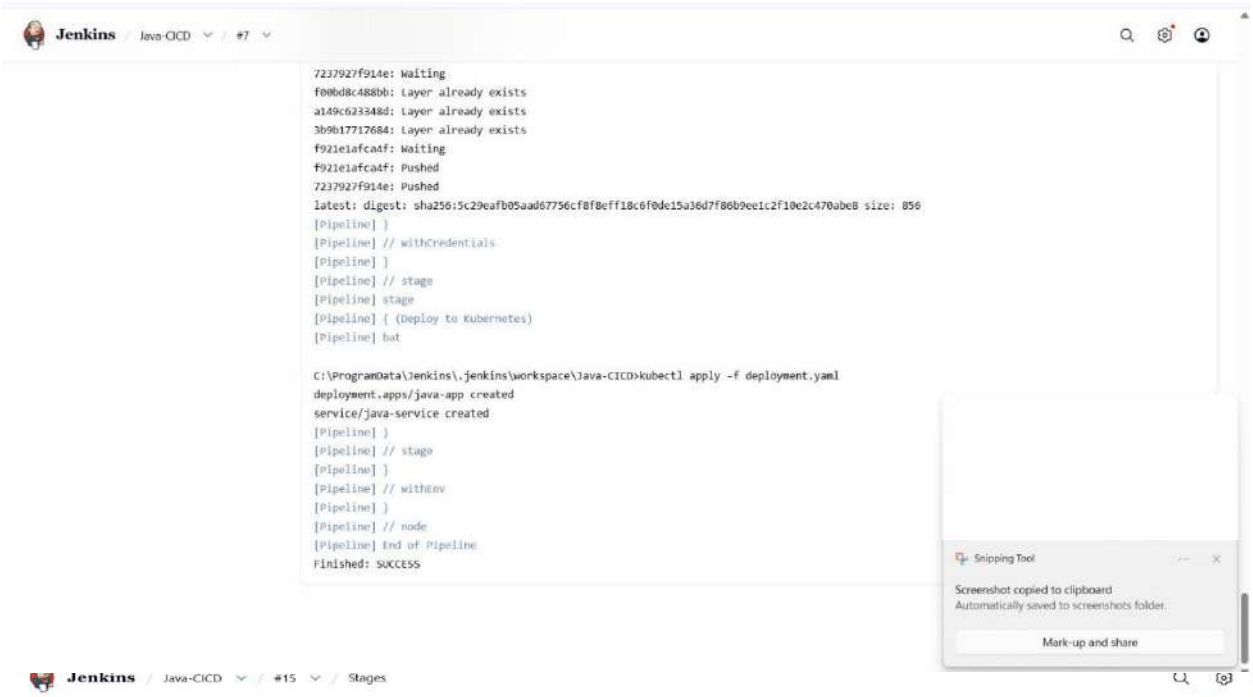
Status
Changes
Console Output
Edit Build Information
Delete build '#15'
Timings
Git Build Data
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build

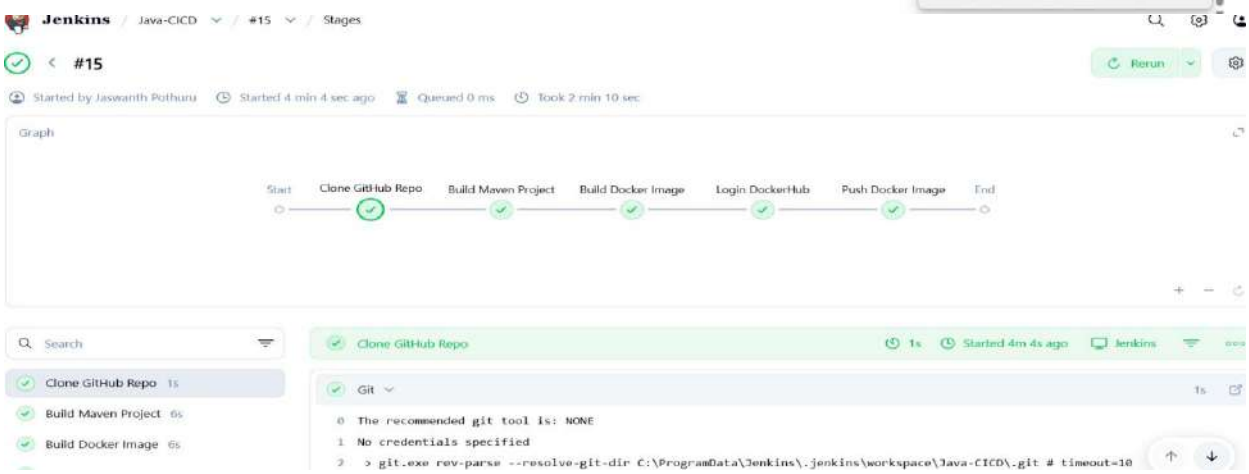
Console

Download Copy View as plain text

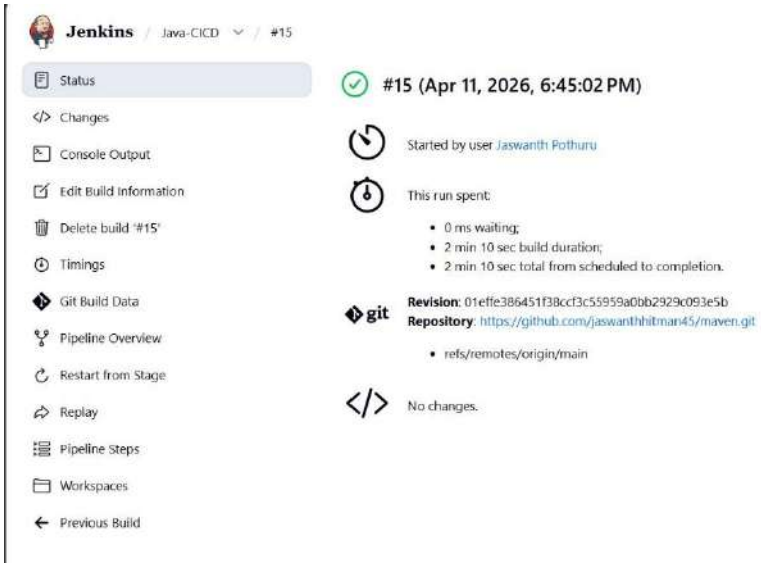
```
Started by user Jaswanth Pothuru
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Java-CICD
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Clone GitHub Repo)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
 > git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\Java-CICD\.git # timeout=10
Fetching changes from the remote Git repository
 > git.exe config remote.origin.url https://github.com/jaswanthhitman45/maven.git # timeout=10
Fetching upstream changes from https://github.com/jaswanthhitman45/maven.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.48.1.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/jaswanthhitman45/maven.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 01effe386451f38ccf3c55959a0bb2929c093e5b (refs/remotes/origin/main)
```

Pipeline Success

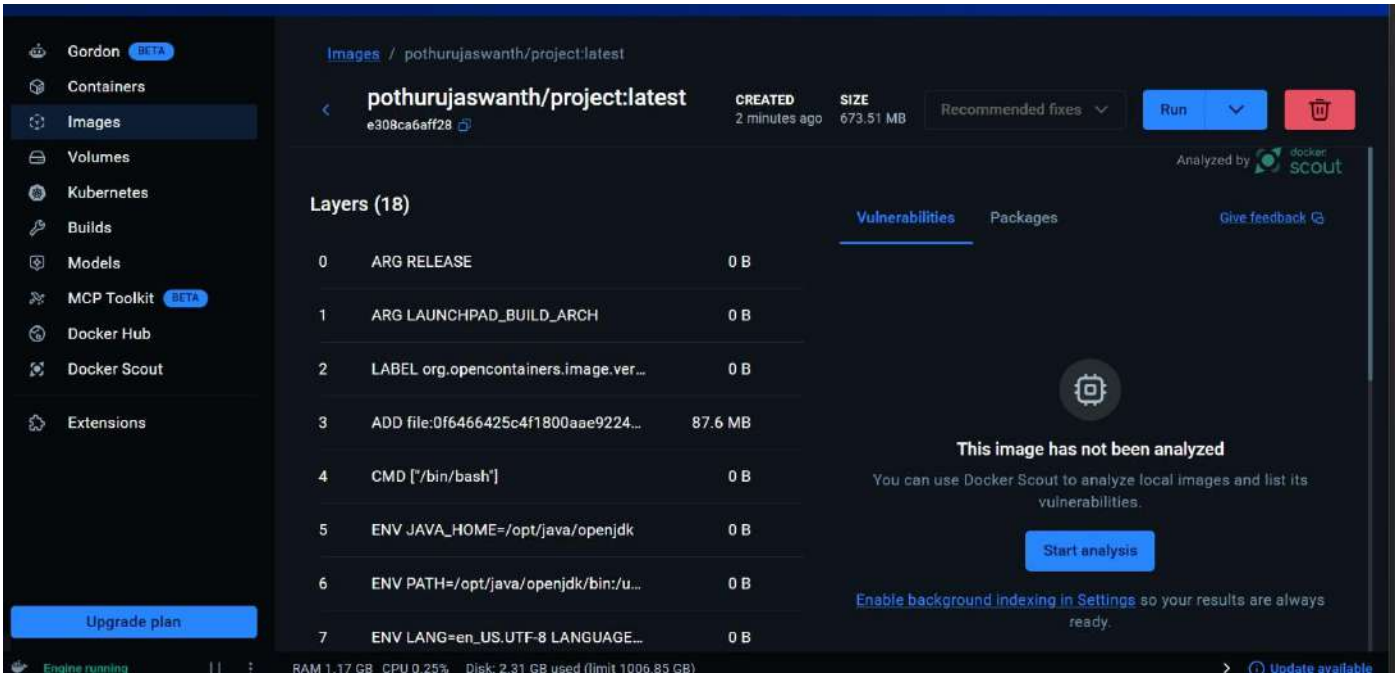




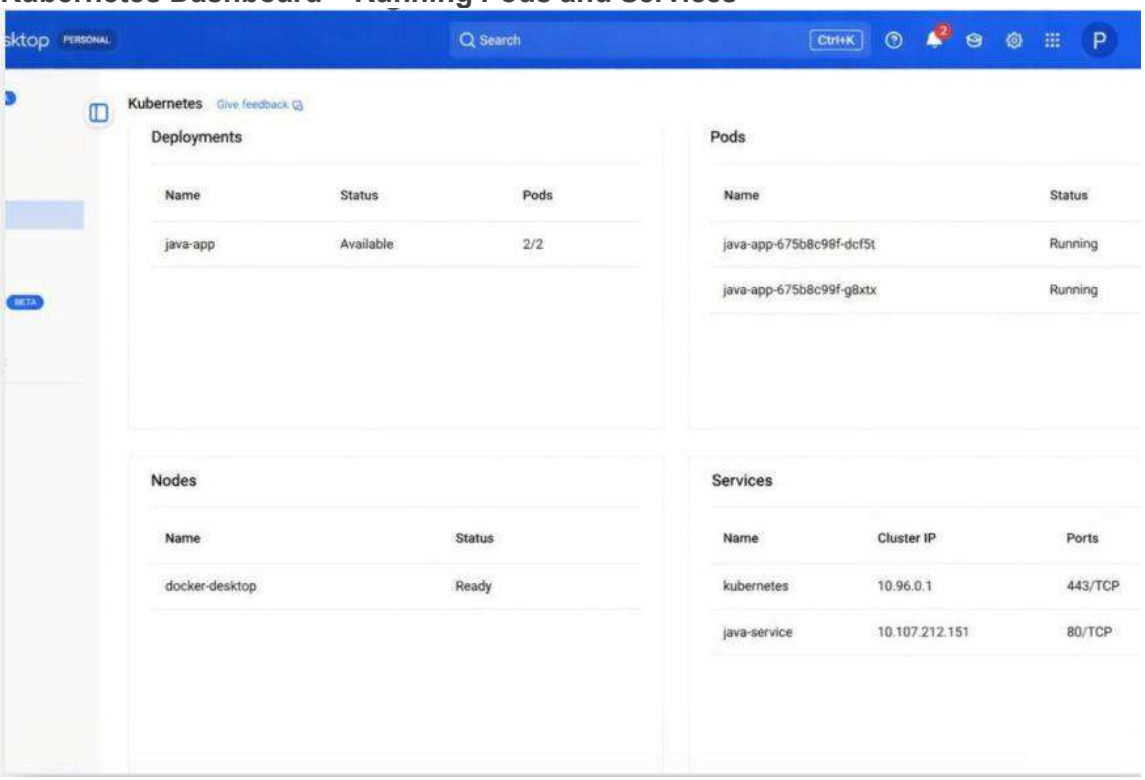
Full Pipeline View



Container Image Stored in Docker Desktop



Kubernetes Dashboard – Running Pods and Services



Kubernetes Pods Status and Logs Output

```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\svmhg>kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
java-app-6765b8c98f-dcf5t     1/1     Running   0          2m13s
java-app-6765b8c98f-g8xtx     1/1     Running   0          2m13s

C:\Users\svmhg>kubectl logs java-app-6765b8c98f-dcf5t
Java CI/CD Application Started...
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
Hello from Java CI/CD Pipeline!
```

Kubernetes Service Configuration (NodePort Exposure)

```
C:\Users\svmhg>kubectl get svc
NAME           TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
java-service   NodePort    10.107.212.151   <none>        80:30008/TCP   17m
kubernetes     ClusterIP   10.96.0.1        <none>        443/TCP        88m

C:\Users\svmhg>kubectl describe deployment java-app
Name:                   java-app
Namespace:              default
CreationTimestamp:      Fri, 10 Apr 2026 22:29:56 +0530
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=java-app
Replicas:               2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=java-app
  Containers:
   java-app:
    Image:        akshayguptha25/java-app:latest
    Port:         <none>
    Host Port:    <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   java-app-6765b8c98f (2/2 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  21m   deployment-controller  Scaled up replica set java-app-6765b8c98f from 0 to 2
```

3. Lab 2: Spring Boot CI/CD Application

3.1 Project Overview

This lab extends the CI/CD pipeline concept to a Spring Boot web application. The application exposes a REST endpoint at the root URL ("/") that returns a confirmation message. The pipeline automates the same stages as Lab 1, with the addition of Spring Boot-specific Maven configuration and a different container port mapping.

The key difference from Lab 1 is the introduction of a web layer (Spring MVC) and the use of Spring Boot's embedded Tomcat server on port 8081.

3.2 Application Source Code

App.java — Spring Boot Entry Point

The main application class bootstraps the Spring context:

```
package com.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

HelloController.java — REST Controller

A simple REST controller exposes a GET endpoint at "/" that returns a status message:

```
package com.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    @GetMapping("/")
    public String home() {
        return "🚀 Spring Boot CI/CD Working!";
    }
}
```

3.3 Maven Build Configuration (pom.xml)

The POM inherits from spring-boot-starter-parent (v3.2.5) and includes the spring-boot-starter-web dependency for the embedded web server and REST support:

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
  </parent>
  <groupId>com.demo</groupId>
  <artifactId>springboot-app</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

3.4 Application Properties

The server port is configured via the application.properties file to run on port 8081 (to avoid conflicts with other services):

```
server.port=8081
```

3.5 Dockerfile

The Dockerfile is similar to Lab 1 but uses ENTRYPOINT instead of CMD, which is the recommended practice for Spring Boot applications:

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY target/springboot-app-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java","-jar","app.jar"]
```

3.6 Kubernetes Deployment (deployment.yaml)

The Kubernetes manifest deploys 1 replica and exposes the service on NodePort 30009, mapping to the container's internal port 8081:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: springboot-app
  template:
    metadata:
      labels:
        app: springboot-app
```

```
    spec:
      containers:
      - name: springboot-app
        image: akshayguptha25/springboot-app:latest
        ports:
        - containerPort: 8081
---
apiVersion: v1
kind: Service
metadata:
  name: springboot-service
spec:
  type: NodePort
  selector:
    app: springboot-app
  ports:
  - port: 80
    targetPort: 8081
    nodePort: 30009
```

3.7 Jenkins Pipeline

The Jenkins pipeline follows the same five-stage pattern as Lab 1, targeting the Spring Boot repository and image:

```
pipeline {
  agent any
  environment {
    DOCKER_USERNAME = "pothurujaswath"
    IMAGE_NAME=”springboot-app”
    IMAGE_TAG=”latest”
  }
  stages {
    stage('Clone Repository') {
      steps {
        git branch: 'main',
            url:  'https://github.com/jaswanthhitman45/maven.git'
      }
    }
    stage('Build with Maven') {
      steps {
        bat 'mvn clean package'
      }
    }
    stage('Build Docker Image') {
      steps {
        bat "docker build -t %DOCKER_IMAGE% ."
      }
    }
    stage('Push to DockerHub') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'dockerhub-cred',
          usernameVariable: 'DOCKER_USER',
          passwordVariable: 'DOCKER_PASS'
        )]) {
          bat """
            docker login -u %DOCKER_USER% -p %DOCKER_PASS%
            docker push %DOCKER_IMAGE%
          """
        }
```

```
    stage('Deploy to Kubernetes') {
      steps {
        bat 'kubectl apply -f deployment.yaml'
      }
    }
  }
}
```

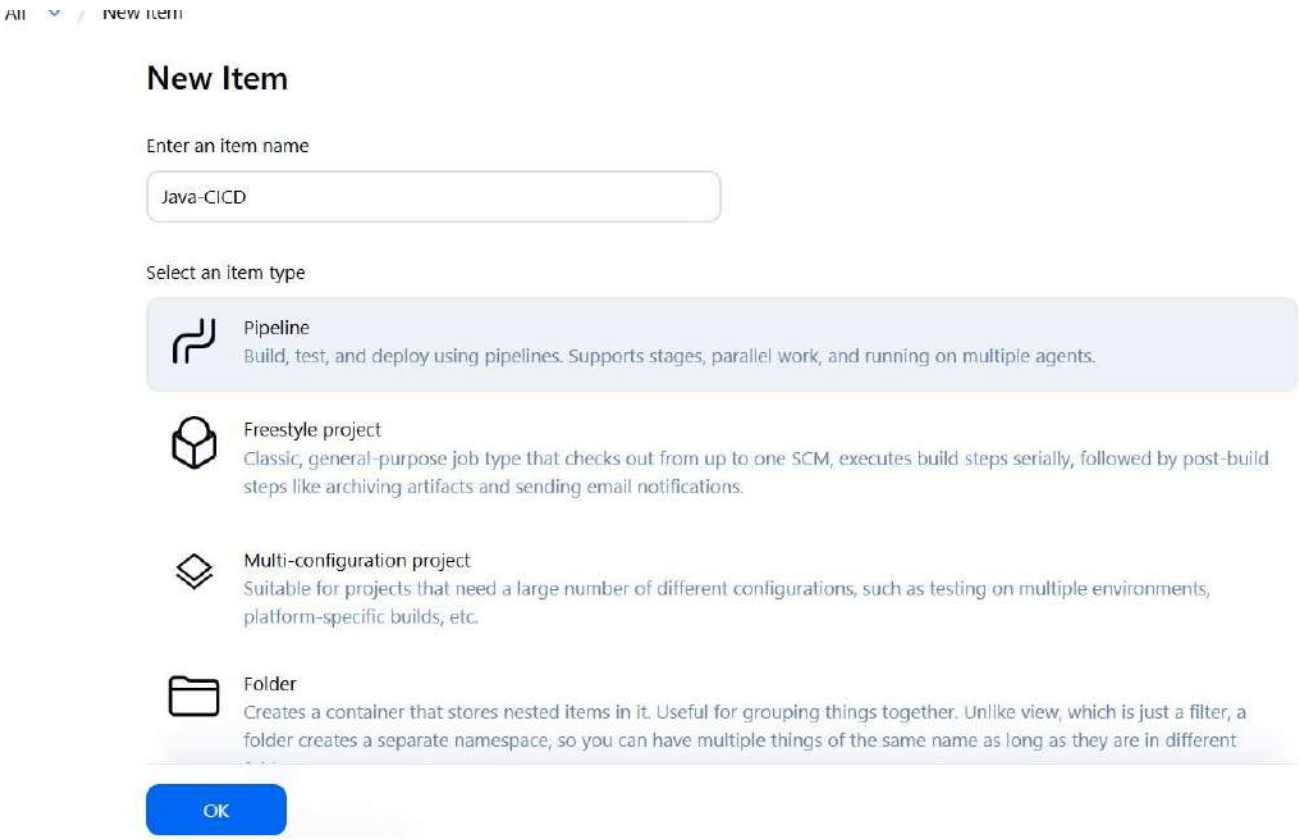
3.8 Pipeline Execution Screenshots

The following screenshots demonstrate the complete execution of the Spring Boot CI/CD pipeline and application deployment.

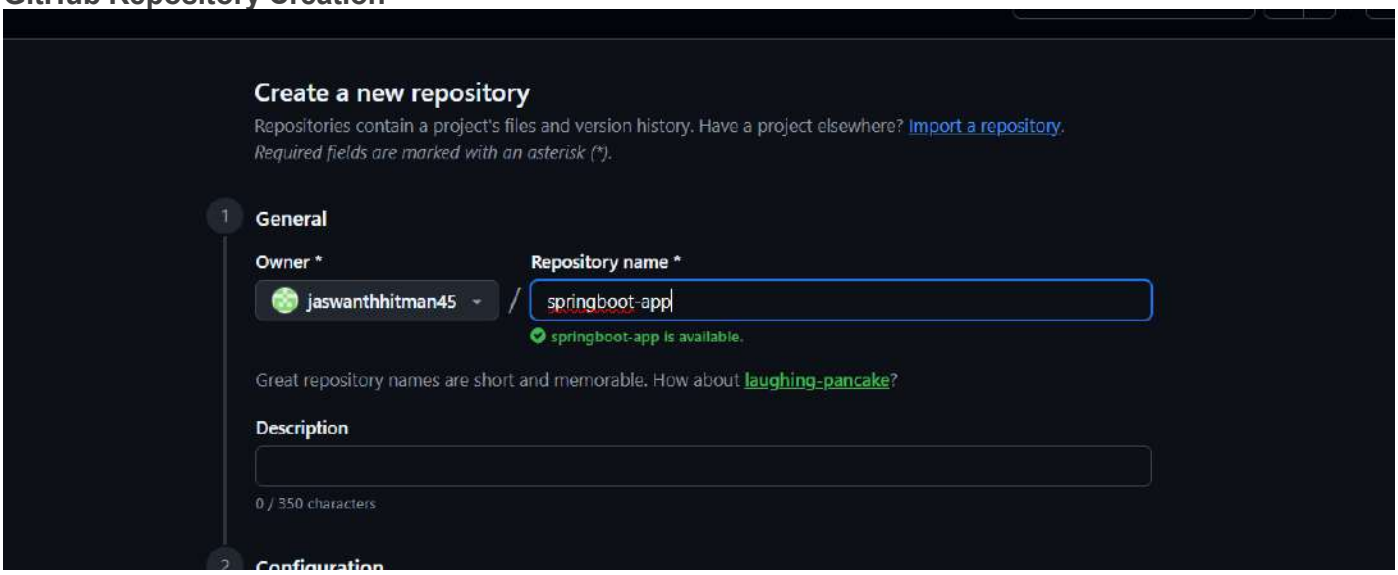
Project Directory Structure (Spring Boot Application)

```
PS C:\Users\svmhg\OneDrive\Desktop\springboot-app> tree /f
Folder PATH listing for volume Windows-SSD
Volume serial number is 36F8-66E8
C:.
    deployment.yaml
    Dockerfile
    pom.xml

    ---src
    +---main
    |   +---java
    |   |   \---com
    |   |       \---demo
    |   |               App.java
    |   |               HelloController.java
    |   |
    |   \---resources
    |           application.properties
    |
    \---target
        |   springboot-app-0.0.1-SNAPSHOT.jar
        |   springboot-app-0.0.1-SNAPSHOT.jar.original
        |
        +---classes
        |   |   application.properties
        |   |
        |   \---com
        |       \---demo
        |               App.class
        |               HelloController.class
        |
        +---generated-sources
        |   \---annotations
        +---maven-archiver
        |   \---pom.properties
        |
        \---maven-status
            \---maven-compiler-plugin
                \---compile
                    \---default-compile
                            createdFiles.lst
                            inputFiles.lst
```

Jenkins Pipeline Creation



GitHub Repository Creation



Jenkins Pipeline Build

Definition

Pipeline script

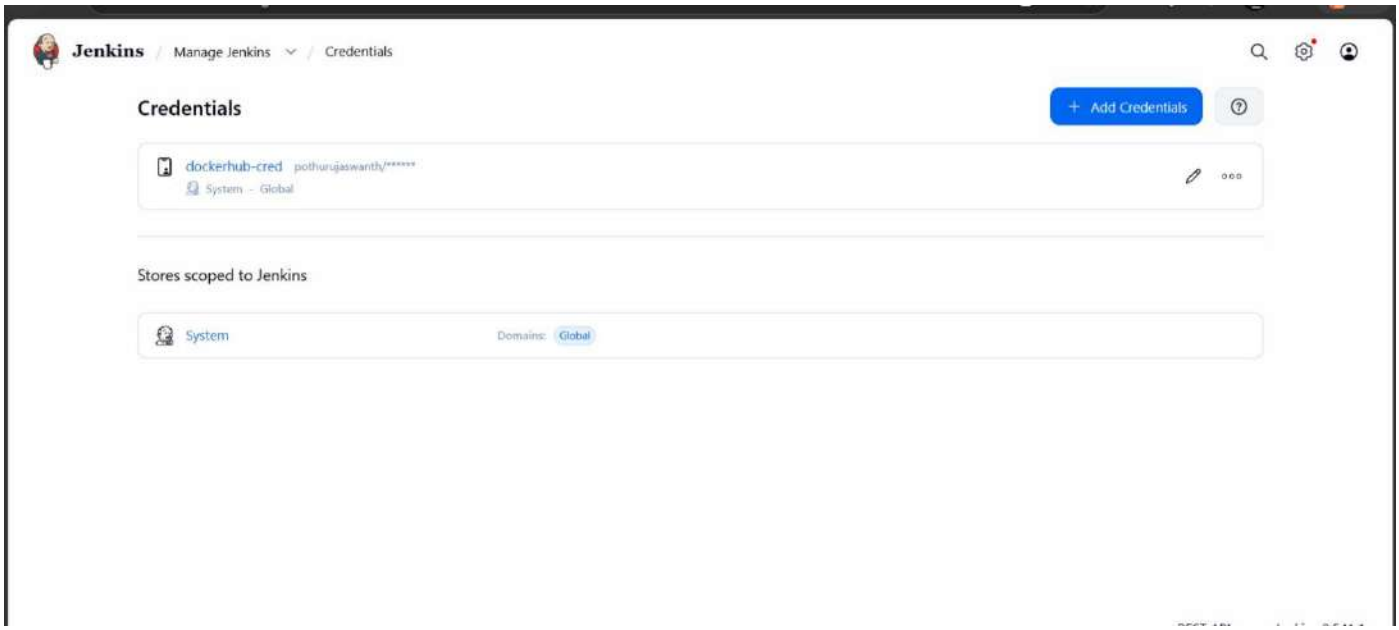
Script ?

try sample Pipeline...

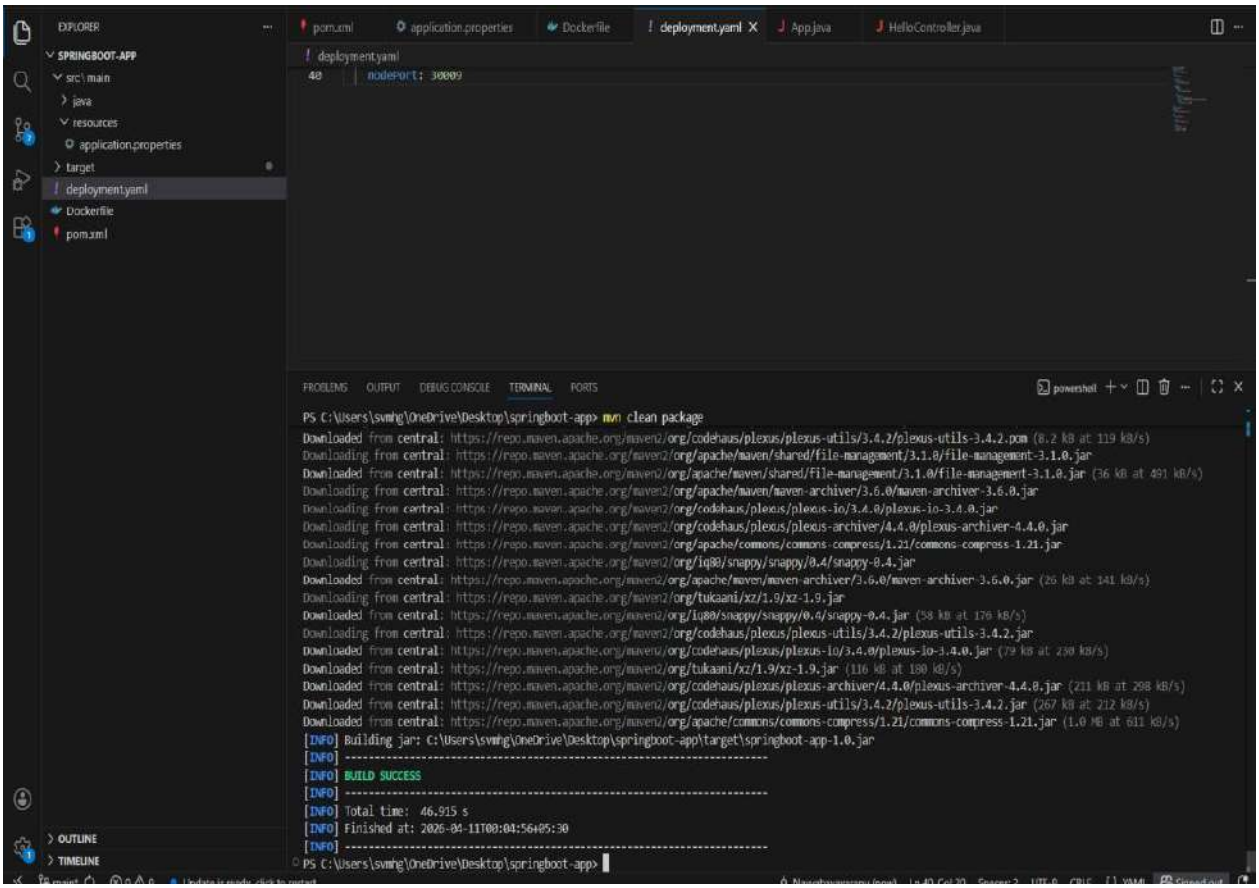
```
    environment {
        DOCKER_USERNAME = "pothurujaswanth"
        IMAGE_NAME = "docker-project"
        IMAGE_TAG = "latest"
    }

    stages {
         stage('Clone GitHub Repo') {
    steps {
        git branch: 'main',
        url: 'https://github.com/jaswanthhitman45/maven.git'
    }
}
```

Save Apply

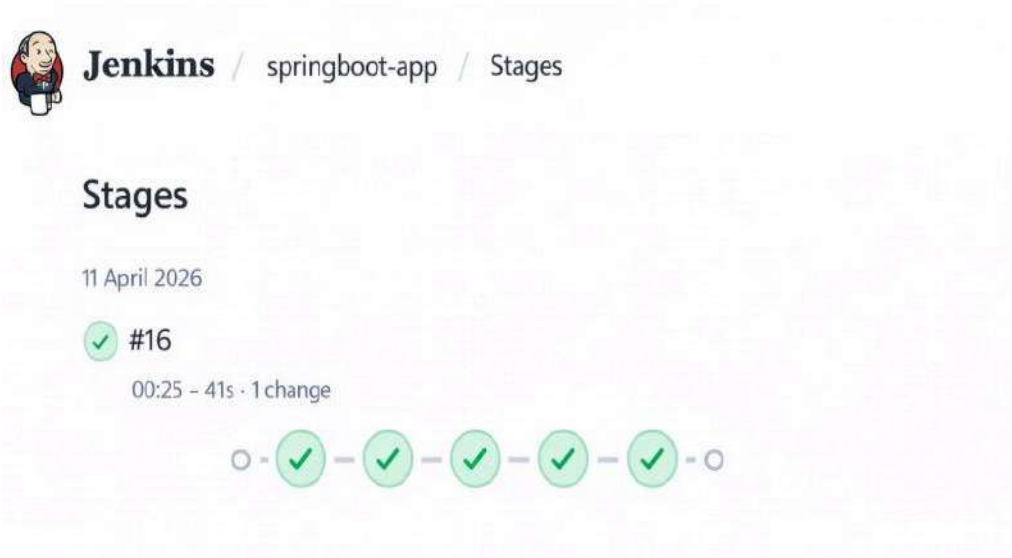
DockerHub Credentials

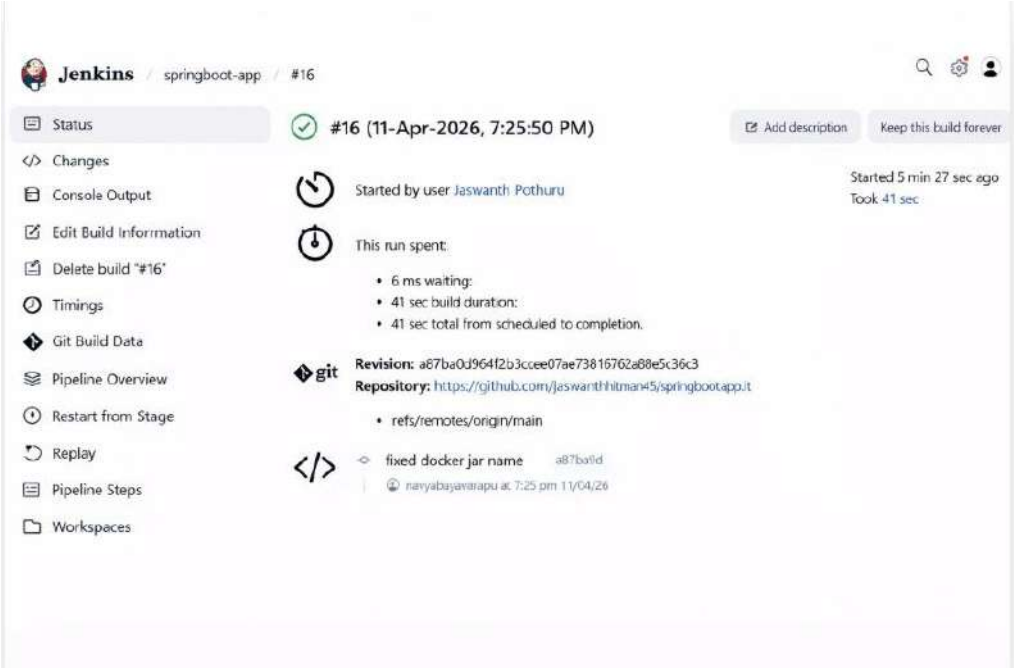


Maven Build Output

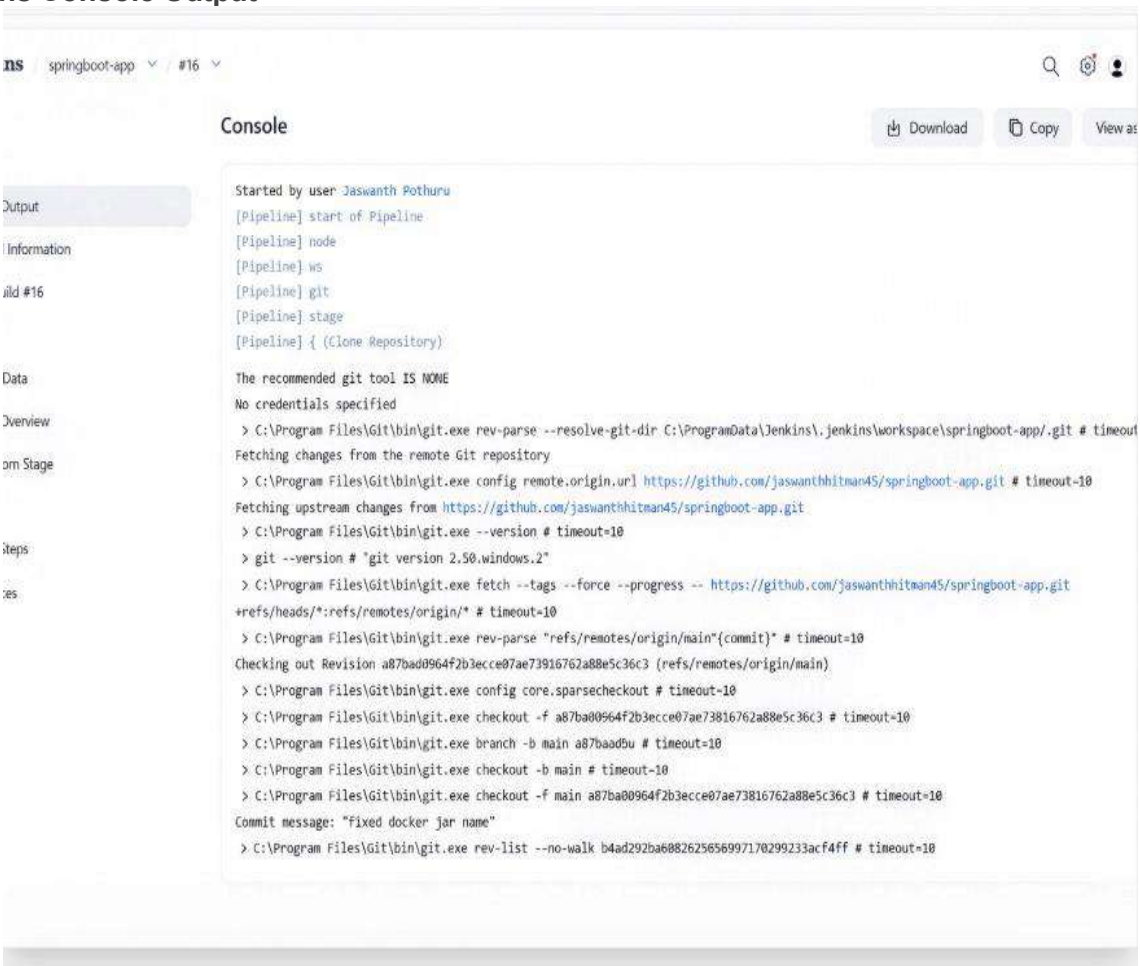


Successful Build and Deployment Stages in Jenkins Pipeline



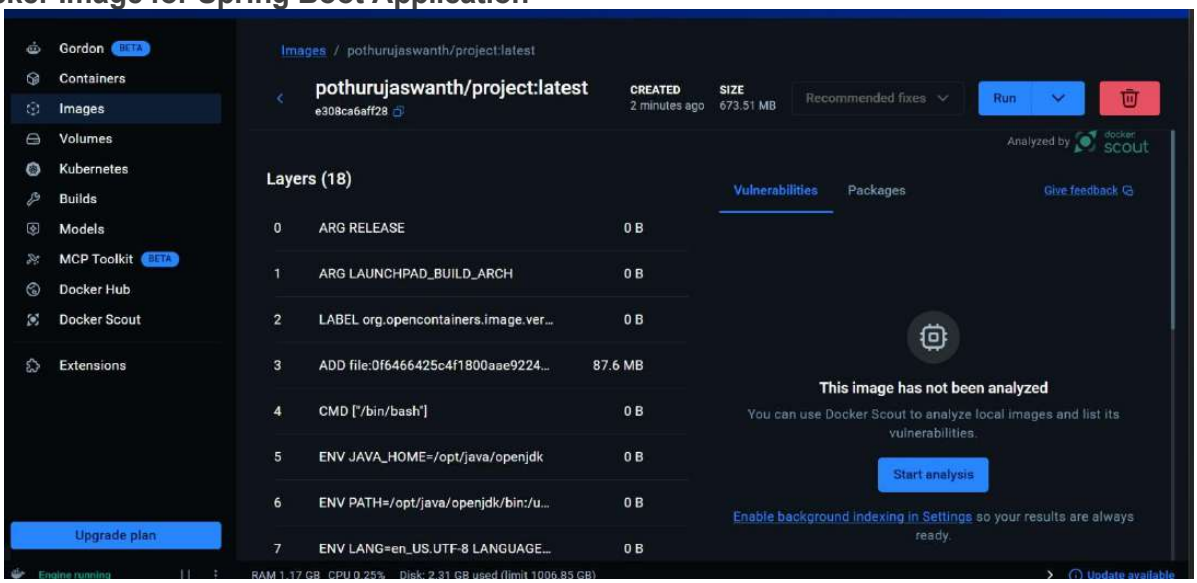


Jenkins Console Output

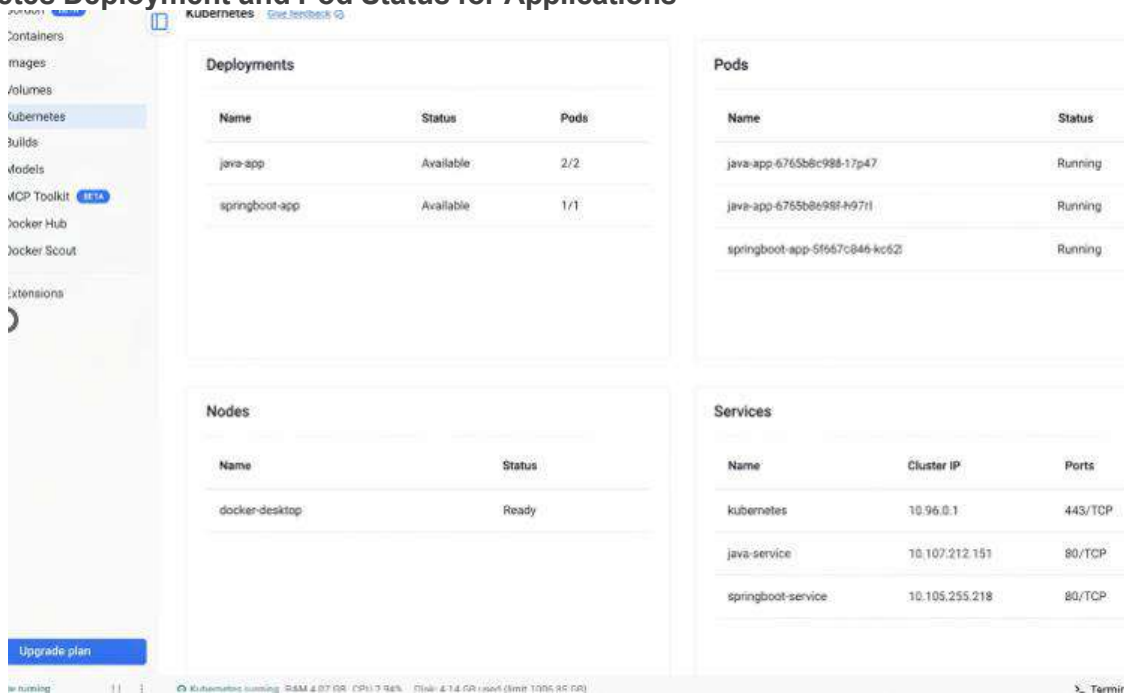


Pipeline Success

Docker Image for Spring Boot Application



Kubernetes Deployment and Pod Status for Applications



Kubernetes Pods Running Successfully

```
C:\Users\svmhg>kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
java-app-6765b8c98f-h97rl         1/1     Running   0          10s
java-app-6765b8c98f-t7p47         1/1     Running   0          10s
springboot-app-5f667c846-kc62l    1/1     Running   0          10s

C:\Users\svmhg>
```

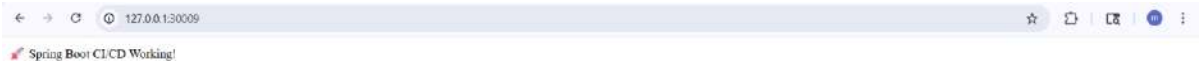
Kubernetes Services and NodePort Configuration

```
C:\Users\svmhg>kubectl get svc
NAME                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
java-service        NodePort    10.107.212.151   <none>        80:30008/TCP   123m
kubernetes          ClusterIP   10.96.0.1        <none>        443/TCP        3h14m
springboot-service  NodePort    10.105.255.218   <none>        80:30009/TCP   33m
```

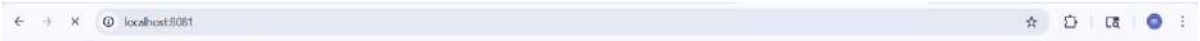
Spring Boot Application Logs in Kubernetes Pod

```
C:\Users\svmhg>kubectl logs springboot-app-5f667c846-kc62l

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.5)

2026-04-18T18:57:18.140Z  INFO 1 --- [           main] com.demo.App                          : Starting App v0.0.1-SNAPSHOT using Java 17.0.18 with PID 1
 (/app/app.jar started by root in /app)
2026-04-18T18:57:18.143Z  INFO 1 --- [           main] com.demo.App                          : No active profile set, falling back to 1 default profile:
"default"
2026-04-18T18:57:18.966Z  INFO 1 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port 8081 (http)
2026-04-18T18:57:18.975Z  INFO 1 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-04-18T18:57:18.976Z  INFO 1 --- [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.20]
2026-04-18T18:57:19.004Z  INFO 1 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2026-04-18T18:57:19.006Z  INFO 1 --- [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 78
1 ms
2026-04-18T18:57:19.250Z  INFO 1 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8081 (http) with context path ''
2026-04-18T18:57:19.263Z  INFO 1 --- [           main] com.demo.App                          : Started App in 1.522 seconds (process running for 1.869)
2026-04-18T18:57:56.615Z  INFO 1 --- [nio-8081-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-04-18T18:57:56.616Z  INFO 1 --- [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2026-04-18T18:57:56.617Z  INFO 1 --- [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms

C:\Users\svmhg>
```

Application Output in Web Browser (NodePort Access)



Spring Boot CI/CD Working!



Spring Boot CI/CD Working!

4. Summary & Conclusion

4.1 What Was Achieved

Both labs successfully demonstrated the implementation of a fully automated CI/CD pipeline using Jenkins, Docker, and Kubernetes. The key achievements are:

• Automated code checkout from GitHub on every pipeline run.

• Reproducible Maven builds producing a portable JAR artifact.

• Containerisation of the application using a Dockerfile with the Eclipse Temurin JDK 17 base image.

• Secure credential management in Jenkins for DockerHub authentication.

• Automated image push to DockerHub, making the image available in a centralised registry.

• Zero-touch deployment to Kubernetes using kubectl apply, updating running pods automatically.

4.2 Comparison of Lab 1 vs Lab 2

Lab 1 focused on a standalone Java application with no web framework, running as a persistent background process. Lab 2 introduced a Spring Boot REST API with an embedded Tomcat server, demonstrating how the same pipeline structure scales to a more complex application with external-facing HTTP endpoints.

4.3 Key Learnings

These labs provided hands-on experience with the core pillars of modern DevOps practice: Infrastructure as Code (Kubernetes manifests), Pipeline as Code (Jenkinsfile), and immutable container-based deployments. The Agile nature of the pipeline allows rapid iteration — any code change pushed to GitHub triggers an automated build, test, and deploy cycle with no manual intervention.